

FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Artificial Intelligence & Data Science · Karachi Campus

PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

Laboratory Manual

LAB 04

Functions and Modular Code

Parameters · Default Arguments · Return Values · Local and Global Scope
zip · enumerate · map · filter · lambda

PREPARED BY

Khadeejah Ashraf

Lab Instructor | Department of Artificial Intelligence & Data Science

Contents

1. Lab Objectives
2. Lab Rules and Conduct
3. Why Functions Exist
4. Defining and Calling a Function
5. Parameters and Arguments
6. Return Values
7. Scope — Where a Name Lives
8. Docstrings and Function Design
9. Functions That Take Other Sequences: `zip` and `enumerate`
10. `lambda` — the Anonymous Function
11. `map()` and `filter()`
12. Putting It Together — A Small Marks Module
13. Common Errors
14. Quick Reference
15. Class Tasks (to be completed during the session)
16. Home Tasks
17. Submission and Assessment

1. Lab Objectives

By the end of this session, you should be able to:

- Explain why a program is broken into functions, and identify repeated code that should become one.
- Define a function with `def`, call it, and describe what happens to control and to memory during the call.
- Pass information in through **positional**, **keyword** and **default** parameters, and state the rules that govern their order.
- Accept a variable number of arguments with `*args` and `**kwargs`.
- Distinguish clearly between `return` and `print()`, and return several values as a tuple.
- Explain **local** and **global** scope, the effect of the `global` keyword, and the order in which Python looks a name up.
- Predict whether a function can change its argument, and explain the difference between passing a list and passing a number.
- Write a docstring and read one with `help()`.
- Use `zip()` and `enumerate()` deliberately, including unzipping with `zip(*pairs)`.
- Write a `lambda` and explain when an anonymous function is appropriate and when a named `def` is better.
- Apply `map()` and `filter()`, and translate freely between them and the equivalent comprehension.

2. Lab Rules and Conduct

- Maintain discipline throughout the lab session.
- Listen carefully and follow the instructions as they are given.
- Raise your hand if you face any problem; do not leave your seat.
- **Class tasks** must be completed and demonstrated during the session itself.
- **Home tasks** must be submitted before the deadline announced on the course portal.
- Get your work checked by the instructor before the end of the session.

Reminder: work that is not checked during the session carries no marks, even if the code runs correctly.

3. Why Functions Exist

Up to now every program you have written has been one long block of statements. That works while a program is small, but it fails in three ways as soon as it grows: the same lines get copied from place to place, a change has to be made in several places at once, and the reader has no way to see the shape of the program without reading every line of it.

A **function** is a named block of statements that you write once and call whenever you need it. Compare the two versions below — they produce identical output.

Without a function. The averaging logic appears three times, and a change to it must be made three times.

```
ali = [78, 45, 90]
sara = [88, 92, 95]
bilal = [65, 70, 58]

print("Ali :", round(sum(ali) / len(ali), 2))
print("Sara :", round(sum(sara) / len(sara), 2))
print("Bilal:", round(sum(bilal) / len(bilal), 2))
```

With a function. The logic exists in exactly one place, and it has a name that says what it is.

```
def average(marks):
    return round(sum(marks) / len(marks), 2)

print("Ali :", average([78, 45, 90]))
print("Sara :", average([88, 92, 95]))
print("Bilal:", average([65, 70, 58]))
```

Output

```
Ali : 71.0
Sara : 91.67
Bilal: 64.33
```

A function gives you	Which means in practice
Reuse	The same code is called from many places instead of being copied.
One place to fix	A bug is corrected once, and every caller is corrected with it.
A name	<code>average(marks)</code> states the intent; the arithmetic behind it can be ignored while reading.
Isolation	Names created inside a function do not leak out and collide with the rest of the program.
Testability	A function that returns a value can be checked against a known answer.

You have already used functions: `print()`, `len()`, `sum()`, `range()` and `input()` are all functions. This lab is about writing your own.

4. Defining and Calling a Function

A definition begins with the keyword `def`, then the function name, then a bracketed list of parameters, then a colon. Everything indented below the colon is the body.

```

    parameters — ndigits has a default
function name
def average(marks, ndigits=2):———— header ends with a colon

    """Return the mean of a list of marks."""—— docstring

    total = sum(marks)———— body — indented four spaces

    return round(total / len(marks), ndigits)—— sends a value back

result = average([78, 45, 90])———— arguments fill the parameters

```

Figure 4.1 The parts of a function definition and of a call

Two things are worth noticing straight away. First, a definition does not run the body — it only creates the function and gives it a name; the body runs when the function is **called**. Second, the call is an expression, so it can be printed, stored in a variable, or used inside a larger expression.

```

def greet(name):
    print("Assalam-o-Alaikum,", name)

print("before the call")
greet("Ali")           # now the body runs
greet("Sara")
print("after the call")

```

Output

```

before the call
Assalam-o-Alaikum, Ali
Assalam-o-Alaikum, Sara
after the call

```

4.1 Definition Before Use

Python reads a file from top to bottom, so a function must be defined before the line that calls it. Calling it earlier raises a `NameError`.

```

greet("Ali")           # NameError: name 'greet' is not defined

def greet(name):
    print("Hello,", name)

```

In a notebook: the rule is the same, but it applies to the **order in which you run the cells**, not the order in which they appear on screen. If a function seems to have vanished, run its cell again — or use **Kernel ► Restart & Run All**.

4.2 Naming a Function

- Use a verb or a verb phrase in lower case with underscores: `calculate_average`, `is_prime`, `count_vowels`.
- A function that answers a yes/no question is conventionally named `is_...` or `has_...` and returns `True` or `False`.
- Do not reuse the name of a built-in. Defining `def sum(...)` hides the real `sum()` for the rest of the program.
- One function should do one job. If you cannot describe it in a single sentence without the word “and”, split it in two.

5. Parameters and Arguments

The names in the definition are **parameters**; the values supplied at the call are **arguments**. The two words are often used loosely, but keeping them apart makes error messages much easier to read.

5.1 Positional Arguments

By default, arguments are matched to parameters by position: the first argument fills the first parameter, and so on. The order is therefore part of the meaning.

```
def power(base, exponent):
    return base ** exponent

print(power(2, 10))      # base = 2, exponent = 10
print(power(10, 2))      # the same arguments, the other way round
```

Output

```
1024
100
```

5.2 Keyword Arguments

An argument may instead be given by name. Keyword arguments may appear in any order, and they document the call at the point where it is written — which is worth a great deal when a function takes several similar values.

```
def power(base, exponent):
    return base ** exponent

print(power(exponent=10, base=2))      # order no longer matters
print(power(2, exponent=10))           # positional first, then keyword
```

Output

```
1024
1024
```

Rule: every positional argument must come before every keyword argument. `power(base=2, 10)` is a `SyntaxError`.

5.3 Default Parameter Values

A parameter may be given a default in the definition. The caller can then leave it out, and the default is used. This is how one function serves both the common case and the special case without a second version of the code.

```
def average(marks, ndigits=2):
    return round(sum(marks) / len(marks), ndigits)

print(average([78, 45, 91]))           # ndigits takes its default, 2
print(average([78, 45, 91], 0))        # supplied positionally
print(average([78, 45, 91], ndigits=4)) # supplied by keyword
```

Output

```
71.33
71.0
71.3333
```

```
def greet(name, greeting="Assalam-o-Alaikum", punctuation="!"):
    return greeting + ", " + name + punctuation

print(greet("Ali"))
print(greet("Sara", "Good morning"))
print(greet("Bilal", punctuation="."))
```

Output

```
Assalam-o-Alaikum, Ali!
Good morning, Sara!
Assalam-o-Alaikum, Bilal.
```

Rule: parameters with defaults must come **after** those without. `def f(a=1, b)` is a `SyntaxError`, because Python would have no way to know which value `f(5)` was meant to fill.

5.4 The Mutable Default Trap

A default value is evaluated **once**, when the function is defined — not on every call. If the default is a list or a dictionary, every call that relies on it shares the same object, and the surprises accumulate.

```
def add_mark(mark, marks=[]):          # dangerous
    marks.append(mark)
    return marks

print(add_mark(50))
print(add_mark(60))                    # the same list is still there
```

Output

```
[50]
[50, 60]
```

The fix is to default to `None` and create a fresh list inside the function:

```
def add_mark(mark, marks=None):        # correct
    if marks is None:
        marks = []
    marks.append(mark)
    return marks

print(add_mark(50))
print(add_mark(60))
```

Output

```
[50]
[60]
```

5.5 Any Number of Arguments: `*args` and `**kwargs`

A parameter written `*args` collects every remaining positional argument into a **tuple**, and `**kwargs` collects every remaining keyword argument into a **dictionary**. The names are only a convention — the `*` and `**` do the work.

```
def total(*marks):
    print(type(marks), marks)
    return sum(marks)

print(total(78, 45))
print(total(78, 45, 90, 62))
print(total())
```

Output

```
<class 'tuple'> (78, 45)
123
<class 'tuple'> (78, 45, 90, 62)
275
<class 'tuple'> ()
0
```

```
def show_record(roll, **details):
    print("Roll:", roll)
    for field, value in details.items():
        print("  ", field, "=", value)

show_record("22K-1234", name="Ali", cgpa=3.4, section="BAI-1A")
```

Output

```
Roll: 22K-1234
  name = Ali
  cgpa = 3.4
  section = BAI-1A
```

The `*` also works at the **call**, where it does the opposite job — it spreads a sequence out into separate arguments:

```
def power(base, exponent):
    return base ** exponent

values = [2, 10]
print(power(*values))          # the same as power(2, 10)
```

Output

```
1024
```

6. Return Values

6.1 return Is Not print

This is the single most common misunderstanding in this lab. `print()` puts characters on the screen for a human to read; `return` hands a **value** back to the code that made the call, so the program can go on using it. A function that prints its answer instead of returning it cannot be reused inside a larger calculation.

```
def average_printed(marks):
    print(sum(marks) / len(marks))    # shows the answer, returns nothing

def average_returned(marks):
    return sum(marks) / len(marks)    # hands the answer back

a = average_printed([78, 45, 90])    # prints 71.0
b = average_returned([78, 45, 90])    # prints nothing

print("a =", a)
print("b =", b)
print("b doubled:", b * 2)
# print("a doubled:", a * 2)          -> TypeError: unsupported operand type(s) for *:
# 'NoneType' and 'int'
```

Output

```
71.0
a = None
b = 71.0
b doubled: 142.0
```

Remember: a function without a `return` statement returns `None`. If your calculation gives `None`, look for a missing `return` before looking anywhere else.

6.2 Returning Several Values

A `return` may list several values separated by commas. They are packed into a tuple, which the caller usually unpacks straight away — the technique you met with tuples in Lab 03.

```
def statistics(marks):
    return len(marks), sum(marks), sum(marks) / len(marks)

count, total, mean = statistics([78, 45, 90, 62])
print(count, total, round(mean, 2))

result = statistics([78, 45])          # not unpacked
print(result, type(result))
```

Output

```
4 275 68.75
(2, 123, 61.5) <class 'tuple'>
```

6.3 `return` Ends the Function

Execution leaves the function the moment a `return` runs; nothing after it in that call is executed. An **early return** is often the clearest way to deal with a special case at the top of a function.

```
def safe_average(marks):
    if not marks:                # the empty list is false
        return 0                # leave immediately
    return sum(marks) / len(marks)

print(safe_average([78, 45, 90]))
print(safe_average([]))         # no ZeroDivisionError
```

Output

```
71.0
0
```

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False        # one divisor is enough to decide
    return True                 # only reached if the loop finished

print([n for n in range(1, 20) if is_prime(n)])
```

Output

```
[2, 3, 5, 7, 11, 13, 17, 19]
```

7. Scope — Where a Name Lives

The **scope** of a name is the region of the program in which that name can be seen. Every call creates a fresh workspace, its **local** scope, which holds the parameters and any variable created inside the body. When the call ends, that workspace is thrown away.

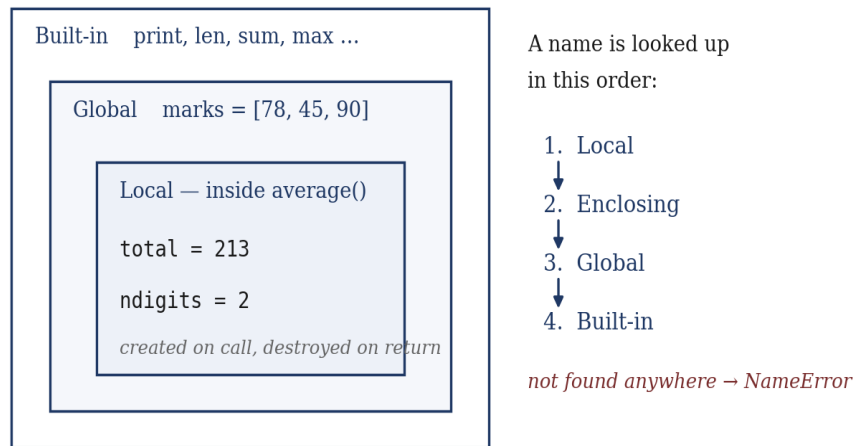


Figure 7.1 Local, global and built-in scope, and the order of lookup

7.1 Local Names Do Not Escape

```
def average(marks):
    total = sum(marks)      # local to this call
    return total / len(marks)

print(average([78, 45, 90]))
print(total)                # NameError: name 'total' is not defined
```

This isolation is a feature, not a restriction. It is what allows two functions written by two different people to use a variable called `total` without interfering with each other.

7.2 Reading a Global Name

A function may **read** a name defined outside it. Python looks in the local scope first and, finding nothing, moves outwards.

```
PASSING_MARK = 50          # a global constant

def has_passed(mark):
    return mark >= PASSING_MARK  # read from the global scope

print(has_passed(45), has_passed(78))
```

Output

```
False True
```

7.3 Assigning Makes a Name Local

Assignment is what creates a local name. If a function assigns to a name anywhere in its body, that name is local **for the whole body** — even on the lines above the assignment.

```
count = 0

def register():
    count = count + 1      # UnboundLocalError
    return count

register()
```

Output

```
UnboundLocalError: cannot access local variable 'count' where it is not associated with a value
```

There are two ways out, and one of them is much better than the other. The `global` keyword tells Python that the assignment refers to the outer name:

```
count = 0

def register():
    global count          # works, but avoid where possible
    count = count + 1
    return count

register(); register()
print(count)
```

Output

2

The preferred alternative is to take the value in as an argument and hand the new value back:

```
def register(count):
    return count + 1      # no hidden dependency on anything outside

count = 0
count = register(count)
count = register(count)
print(count)
```

Output

2

Why avoid global: a function that changes a global variable can be understood only by reading the whole program, and two such functions can quietly break each other. Pass values in, return values out.

7.4 Shadowing

A local name may deliberately or accidentally hide an outer one of the same name. The outer value is untouched — it simply cannot be seen from inside.

```
marks = [78, 45, 90]      # global

def rescale(marks):       # this parameter shadows the global name
    marks = [m + 5 for m in marks]
    return marks

print(rescale(marks))
print(marks)              # the global list is unchanged
```

Output

[83, 50, 95]
[78, 45, 90]

7.5 Can a Function Change Its Argument?

Arguments are passed by **object reference**: the parameter becomes another name for the very same object. What follows depends entirely on whether that object can be changed at all. Numbers, strings and tuples are immutable, so a function can never alter the caller's value. Lists, dictionaries and sets are mutable, so a method call such as `append()` inside the function is visible outside it.

```
def try_to_change(number, items):
    number = number + 100    # rebinds the local name only
    items.append(100)        # changes the object the caller passed
```

```
n = 1
L = [1]
try_to_change(n, L)
print(n)      # unchanged - an int cannot be modified
print(L)      # changed - the list was modified in place
```

Output

```
1
[1, 100]
```

If the function...	Effect on the caller's data
Assigns to the parameter (<code>items = [...]</code>)	None — only the local name is rebound
Calls a mutating method (<code>items.append(x)</code>)	The caller sees the change
Receives an int, float, string or tuple	The caller can never see a change

Design advice: decide deliberately which of the two your function is. Either it returns a new value and leaves its arguments alone, or it modifies an argument in place and returns `None` — as `sort()` does. Doing both at once confuses every reader.

8. Docstrings and Function Design

A **docstring** is a string literal placed as the first line of the function body. It is not a comment: Python keeps it, `help()` prints it, and editors show it while you type the call.

```
def average(marks, ndigits=2):
    """Return the mean of a list of marks, rounded to ndigits places.

    marks    -- a non-empty list of numbers
    ndigits  -- how many decimal places to keep (default 2)
    """
    return round(sum(marks) / len(marks), ndigits)

help(average)
print(average.__doc__.splitlines()[0])
```

Output

```
Help on function average in module __main__:

average(marks, ndigits=2)
    Return the mean of a list of marks, rounded to ndigits places.

    marks    -- a non-empty list of numbers
    ndigits  -- how many decimal places to keep (default 2)

Return the mean of a list of marks, rounded to ndigits places.
```

A useful docstring says what the function **returns**, not how it works. Write it in the imperative — “Return the mean...”, not “This function will calculate...”.

Design rule	In practice
One job per function	If the name needs an “and”, write two functions.

Design rule	In practice
Return rather than print	Printing is the caller's decision, not the function's.
Few parameters	Three or four is plenty; a long list usually means the function does too much.
No hidden dependencies	Everything the function needs arrives through its parameters.
Same input, same output	A function you can predict is a function you can test.

9. Functions That Take Other Sequences: `zip` and `enumerate`

You used `zip()` and `enumerate()` as loop helpers in Lab 02. Now that functions are the subject, it is worth looking at what they actually **are**: ordinary functions that take a sequence and return a new iterable object. They are lazy — the result is produced item by item as you loop over it — which is why printing one directly shows an object rather than a list.

```
subjects = ['Physics', 'Chemistry', 'Maths']
marks    = [78, 45, 90]

print(zip(subjects, marks))           # a lazy object, not a list
print(list(zip(subjects, marks)))     # force it into a list to look inside
print(dict(zip(subjects, marks)))
print(list(enumerate(subjects, start=1)))
```

Output

```
<zip object at 0x7f2c1c0d2b80>
[('Physics', 78), ('Chemistry', 45), ('Maths', 90)]
{'Physics': 78, 'Chemistry': 45, 'Maths': 90}
[(1, 'Physics'), (2, 'Chemistry'), (3, 'Maths')]
```

Once only: a lazy object is exhausted after one pass. Loop over a `zip()` twice and the second loop sees nothing. Wrap it in `list()` if you need it more than once.

9.1 Unzipping with `zip(*pairs)`

The `*` that spreads a sequence into separate arguments (Section 5.5) turns `zip()` into its own inverse: given a list of pairs, `zip(*pairs)` gives back the two original sequences.

```
students = [('Ali', 78), ('Sara', 92), ('Bilal', 65)]

names, marks = zip(*students)
print(names)
print(marks)
print("class average:", round(sum(marks) / len(marks), 2))
```

Output

```
('Ali', 'Sara', 'Bilal')
(78, 92, 65)
class average: 78.33
```

9.2 Passing a Function to a Function

In Python a function is itself a value: it can be stored in a variable, put in a list, and passed to another function. Writing the name **without** brackets refers to the function; adding brackets calls it. This is the idea that `sorted()`, `map()` and `filter()` are built on.

```
def double(x):
```

```

    return x * 2

operation = double          # no brackets - the function itself
print(operation)
print(operation(21))        # brackets - now it runs

def apply_twice(func, value):
    return func(func(value))

print(apply_twice(double, 5))

```

Output

```

<function double at 0x7f2c1c0e5760>
42
20

```

`sorted()` uses exactly this mechanism through its `key` parameter: you hand it a function, and it calls that function once per item to decide what to sort on.

```

words = ['comprehension', 'ai', 'python', 'lab']

print(sorted(words))          # alphabetical
print(sorted(words, key=len)) # by length - len is passed, not called
print(max(words, key=len))

```

Output

```

['ai', 'comprehension', 'lab', 'python']
['ai', 'lab', 'python', 'comprehension']
comprehension

```

10. lambda — the Anonymous Function

A `lambda` is a function written as a single expression, with no name and no `return` keyword — the value of the expression is returned automatically. It exists for the case where a function is needed **once**, as an argument to something else, and giving it a name would add nothing.

```

# these two definitions are equivalent
def double(x):
    return x * 2

double_lambda = lambda x: x * 2

print(double(21), double_lambda(21))

area = lambda length, width: length * width # more than one parameter
print(area(3, 4))

```

Output

```

42 42
12

```

	<code>def</code>	<code>lambda</code>
Has a name	Yes	No, unless you assign it to one
Body	Any number of statements	Exactly one expression
<code>return</code>	Written explicitly	Implicit — the expression is the result
Docstring	Yes	No

	def	lambda
Best used for	Anything you will call by name	A one-line rule passed to another function

The natural home of a lambda is the key parameter of `sorted()`, `max()` and `min()`, where the rule is short and belongs at the point of use:

```
students = [('Ali', 78), ('Sara', 92), ('Bilal', 65)]

print(sorted(students, key=lambda pair: pair[1]))           # by marks
print(sorted(students, key=lambda pair: pair[1], reverse=True)) # best first
print(max(students, key=lambda pair: pair[1]))

records = [{'name': 'Ali', 'cgpa': 3.4}, {'name': 'Sara', 'cgpa': 3.9}]
print(sorted(records, key=lambda r: r['cgpa'], reverse=True))
```

Output

```
[('Bilal', 65), ('Ali', 78), ('Sara', 92)]
[('Sara', 92), ('Ali', 78), ('Bilal', 65)]
('Sara', 92)
[{'name': 'Sara', 'cgpa': 3.9}, {'name': 'Ali', 'cgpa': 3.4}]
```

Style: assigning a lambda to a name — `double = lambda x: x * 2` — gives you all the limitations of a lambda and none of the benefits of a `def`. If it deserves a name, write `def`.

11. map() and filter()

`map()` applies a function to every item of a sequence and yields the results. `filter()` applies a function that returns `True` or `False` and yields only the items for which it was true. Both are lazy, so wrap them in `list()` to see the outcome.

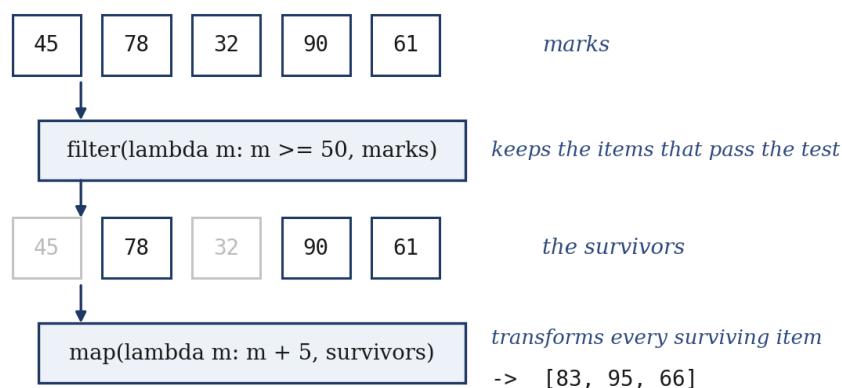


Figure 11.1 A filter followed by a map, over a list of marks

```
marks = [45, 78, 32, 90, 61]

passed = list(filter(lambda m: m >= 50, marks))
bumped = list(map(lambda m: m + 5, passed))
squares = list(map(lambda n: n ** 2, [1, 2, 3, 4]))

print(passed)
print(bumped)
```

```
print(squares)
```

Output

```
[78, 90, 61]
[83, 95, 66]
[1, 4, 9, 16]
```

`map()` also accepts several sequences, and then the function must take one argument from each — the same pairing rule as `zip()`:

```
obtained = [78, 45, 90]
totals    = [100, 50, 100]

percentages = list(map(lambda got, out_of: round(got / out_of * 100, 1),
                       obtained, totals))
print(percentages)
```

Output

```
[78.0, 90.0, 90.0]
```

11.1 map, filter or a Comprehension?

Everything `map()` and `filter()` do can be written as a comprehension, and in most Python code the comprehension is preferred because it reads left to right in one line. Learn both: you will meet `map` and `filter` constantly in existing code and in the data-processing libraries later in this course.

Task	With map / filter	As a comprehension
Transform every item	<code>list(map(f, L))</code>	<code>[f(x) for x in L]</code>
Keep some items	<code>list(filter(p, L))</code>	<code>[x for x in L if p(x)]</code>
Both at once	<code>list(map(f, filter(p, L)))</code>	<code>[f(x) for x in L if p(x)]</code>
Apply a named function	<code>list(map(is_prime, L))</code>	<code>[is_prime(x) for x in L]</code>

```
marks = [45, 78, 32, 90, 61]

print(list(map(lambda m: m + 5, filter(lambda m: m >= 50, marks))))
print([m + 5 for m in marks if m >= 50])          # identical, and easier to read
```

Output

```
[83, 95, 66]
[83, 95, 66]
```

12. Putting It Together — A Small Marks Module

The example below is the shape your Class Tasks should take: several short functions, each with one job and a docstring, and a `main()` that does nothing but call them in order. Notice that no function prints anything except `report()`, which is the one whose job **is** to print.

```
def average(marks, ndigits=2):
    """Return the mean of a list of marks."""
    if not marks:
        return 0
    return round(sum(marks) / len(marks), ndigits)

def grade(mark):
```

```

"""Return the letter grade for one mark."""
if mark >= 80:
    return 'A'
elif mark >= 70:
    return 'B'
elif mark >= 60:
    return 'C'
elif mark >= 50:
    return 'D'
return 'F'

def passed(records):
    """Return only the records whose average is at least 50."""
    return list(filter(lambda r: average(r['marks']) >= 50, records))

def ranked(records):
    """Return the records sorted from the highest average to the lowest."""
    return sorted(records, key=lambda r: average(r['marks']), reverse=True)

def report(records):
    """Print one line per student, best first."""
    for position, record in enumerate(ranked(records), start=1):
        avg = average(record['marks'])
        print(position, record['name'], avg, grade(avg))

def main():
    students = [
        {'name': 'Ali', 'marks': [78, 45, 90]},
        {'name': 'Sara', 'marks': [88, 92, 95]},
        {'name': 'Bilal', 'marks': [40, 38, 52]},
    ]
    report(students)
    print("passed:", [r['name'] for r in passed(students)])
    print("class average:", average([average(r['marks']) for r in students]))

main()

```

Output

```

1 Sara 91.67 A
2 Ali 71.0 B
3 Bilal 43.33 F
passed: ['Ali', 'Sara']
class average: 68.67

```

13. Common Errors

Message or symptom	Usual cause
NameError: name 'f' is not defined	The function is called before its def has run — check the order of your cells.
TypeError: f() missing 1 required positional argument	A parameter was left out of the call and has no default.

Message or symptom	Usual cause
<code>TypeError: f() takes 2 positional arguments but 3 were given</code>	One argument too many — or <code>self</code> is missing in a class (Lab 05).
<code>SyntaxError: positional argument follows keyword argument</code>	A positional argument was written after a keyword one.
<code>UnboundLocalError</code>	The body assigns to a name that also exists globally; pass it in as a parameter instead.
The result is <code>None</code>	A missing <code>return</code> — the function printed its answer instead of returning it.
A default list keeps growing	A mutable default argument; use <code>None</code> and create the list inside.
<code><map object at 0x...></code> is printed	<code>map()</code> and <code>filter()</code> are lazy; wrap the call in <code>list()</code> .

14. Quick Reference

Form	Meaning	Example
<code>def f(a, b):</code>	Define a function of two parameters	<code>def add(a, b): return a + b</code>
<code>f(1, 2)</code>	Positional call	<code>add(1, 2)</code>
<code>f(b=2, a=1)</code>	Keyword call	<code>add(b=2, a=1)</code>
<code>def f(a, b=10):</code>	Default value for <code>b</code>	<code>round(x, ndigits=2)</code>
<code>def f(*args):</code>	Any number of positional arguments (a tuple)	<code>def total(*marks):</code>
<code>def f(**kwargs):</code>	Any number of keyword arguments (a dict)	<code>def show(**details):</code>
<code>f(*values)</code>	Spread a sequence into arguments	<code>power(*[2, 10])</code>
<code>return a, b</code>	Return several values as a tuple	<code>count, total = stats(L)</code>
<code>global name</code>	Assign to a global from inside a function	Avoid where possible
<code>"""..."""</code>	Docstring, read by <code>help()</code>	First line of the body
<code>lambda x: x * 2</code>	A one-expression anonymous function	<code>key=lambda r: r['cgpa']</code>
<code>map(f, L)</code>	Apply <code>f</code> to every item	<code>list(map(str, L))</code>
<code>filter(p, L)</code>	Keep the items where <code>p</code> is true	<code>list(filter(is_prime, L))</code>
<code>sorted(L, key=f)</code>	Sort by the value <code>f</code> returns	<code>sorted(L, key=len)</code>
<code>zip(a, b)</code> <code>zip(*pairs)</code>	Pair up two sequences / split them again	<code>names, marks = zip(*students)</code>

15. Class Tasks

These five tasks are to be completed **during the lab session** and demonstrated to the instructor before you leave. Work in the accompanying notebook `Lab04_Functions.ipynb`, in the section titled **Class Tasks**. Every function you write must have a docstring, and no function except the reporting one may call `print()`.

C1. A Statistics Toolkit — definitions, arguments and return values. Write four functions that each take a list of numbers and **return** (never print) their answer: `total_of(numbers)`, `average_of(numbers, ndigits=2)`, `largest_of(numbers)` — written with a loop, not `max()` — and `count_even(numbers)`. Then write `summary(numbers)` which calls the other four and returns all four results as a single tuple. Finally, call `summary()` on two different lists, unpack the returned tuple into four named variables, and print a tidy report. Every function must return `0` for an empty list instead of raising an error.

Demonstrate `average_of` being called three ways: with the default, with the second argument positionally, and with it as a keyword.

C2. Flexible Arguments — defaults, args and *kwargs. Write `make_result_card(name, roll, *marks, passing=50, **details)` which returns (not prints) a multi-line string containing: the student's name and roll number; every mark passed in, numbered with `enumerate()`; the total, the average and how many marks were below passing; the verdict "PASS" or "FAIL"; and any extra fields supplied as keyword arguments, one per line.

Call it at least four times to show every feature: with two marks, with five marks, with `passing` overridden, and with two extra keyword details such as `section` and `semester`. Print each returned string.

C3. Scope Investigation — predict, then run. For each of the five snippets given in the notebook, first write your prediction as a comment, then run the cell and record what actually happened and why in one sentence. The snippets cover: a local name read after the call; a global read from inside a function; an assignment producing `UnboundLocalError`; the same code fixed with `global`; and a function that receives both a number and a list and tries to change each.

Finish by writing two versions of the same operation — `add_bonus_copy(marks, bonus)` which returns a **new** list and leaves the caller's list alone, and `add_bonus_inplace(marks, bonus)` which modifies the caller's list and returns `None`. Show the difference on the same input, and state in a comment which of the two you would prefer in a library and why.

C4. Rankings with lambda, map and filter. A list of student dictionaries with the keys `name`, `marks` (a list of three) and `section` is given in the notebook. Using `lambda` together with `map()`, `filter()` and `sorted()` — no ordinary loops in this task — produce and print: the name and average of every student, best first; only the students of section 'A'; the names of students whose average is at least 60; every average converted to a percentage string such as '71.0%'; and the single top student, found with `max()` and a key.

Then rewrite any two of the five results as list comprehensions, print them beside the originals to prove they agree, and say in a comment which version you find easier to read.

C5. A Text Toolkit — a module of small functions. Write five functions, each doing one job and each with a docstring: `clean(text)` returning the text in lower case with punctuation removed; `word_count(text)` returning a dictionary of word to frequency; `top_n(counts, n=3)` returning the `n` most frequent (word, count) pairs using `sorted()` with a key; `is_palindrome(word)` returning `True` or `False`; and `report(text)` — the only function allowed to print — which calls the others and displays a small summary. Test `report()` on a sentence typed by the user.

Add at least three `assert` statements that check your functions against answers you have worked out by hand, for example `assert is_palindrome('level')` and `assert word_count('a a b')['a'] == 2`. A failed assertion should be treated as a bug to fix, not a line to delete.

Assessment of class tasks: you must be able to explain any line of your code, including why a particular value is returned rather than printed, and what would change if a `lambda` were replaced by a `def`. A working program that you cannot explain earns no marks.

16. Home Tasks

Attempt all the questions below. Write each program in a **separate .py file**, give every function a docstring, test it with more than one input, and submit it according to the naming convention in Section 17.

- Q1.** Write a function `is_even(n)` that returns `True` or `False`, and use it with `filter()` to print all the even numbers in a list entered by the user. Do not use `%` anywhere except inside the function.
- Q2.** Write `bmi(weight, height)` that returns the body mass index, and `bmi_category(value)` that returns "Underweight", "Normal", "Overweight" or "Obese". Print both results for values entered by the user.
- Q3.** Write `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)`, then use `map()` to convert a whole list of temperatures and print the two lists side by side with `zip()`.
- Q4.** Write `count_vowels(text)` that returns a dictionary mapping each vowel to how many times it appears in the text, and `count_consonants(text)` that returns a single number. Report both for a sentence entered by the user.
- Q5.** Write `factorial(n)` using a loop and `power(base, exponent=2)` with a default exponent. Print a small table of the factorials of 1 to 10 and the squares and cubes of 1 to 5.
- Q6.** Write `statistics(numbers)` that returns the count, the sum, the average, the largest and the smallest as a tuple, computing the largest and smallest with your own loop. Unpack the result at the call and print a report.
- Q7.** Write `apply_to_all(func, items)` which takes a **function** and a list, and returns a new list with the function applied to every item — your own version of `map()`. Test it with a named function, then with a lambda.
- Q8.** Write `grade(mark)` returning a letter grade, and use it with a list comprehension to produce a list of (mark, grade) tuples for marks entered by the user. Then print how many students earned each grade, using a dictionary.
- Q9.** A list of (name, marks) tuples is given. Use `zip(*pairs)` to split it into two sequences, then use `sorted()` with a lambda key to print the students in descending order of marks, and `filter()` to print those above the class average.
- Q10.** Write `word_frequency(sentence)` returning a dictionary of word to count, and `most_common(counts, n=1)` returning the `n` most frequent words. Demonstrate `most_common` being called both with and without its second argument.
- Q11.** Write a function `describe(**details)` that accepts any number of keyword arguments and prints each field on its own line, and a function `total(*numbers)` that accepts any number of positional arguments and returns their sum. Call each of them three times with a different number of arguments.
- Q12.** Demonstrate the mutable default argument trap in your own code: write a function with a `marks=[]` default, call it three times to show the growing list, then write the corrected version using `None` and show that it now behaves. Explain the difference in a comment of no more than two lines.

17. Submission and Assessment

Follow the naming convention below so that your work can be identified and graded correctly:

- **Class tasks:** completed in the notebook `RollNo_Lab04_ClassTasks.ipynb` and demonstrated before the end of the session. Run **Kernel ► Restart & Run All** before you call the instructor.
- **Home tasks:** one script per question, named `RollNo_Lab04_Qn.py` — for example `22K-1234_Lab04_Q7.py`.
- Place all the files for this lab in a single folder and compress it before uploading.
- Every file must begin with a comment giving your name, roll number and the question number, and every function must carry a docstring.
- Code that does not run will not be marked. Test your program before calling the instructor.

Assessment Criterion	Weight
Correctness of output for all valid inputs, including empty input	35%
Class tasks completed and demonstrated in the session	20%
Correct use of parameters, defaults and return values	20%
Modularity: one job per function, no unnecessary globals, docstrings present	15%
In-lab demonstration and understanding of the code	10%

End of Lab 04